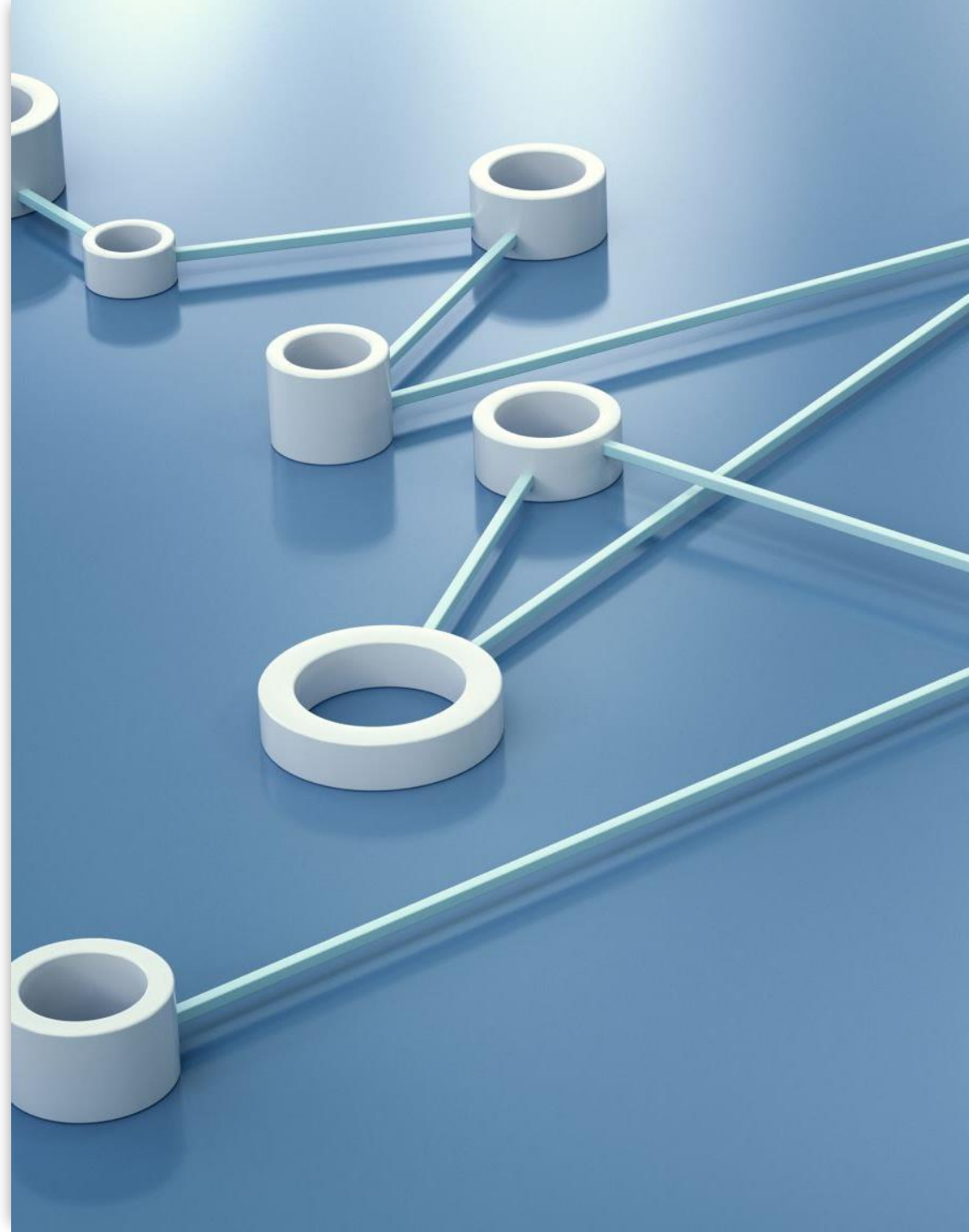


Integrity Constraint

Er. Piyush Pant

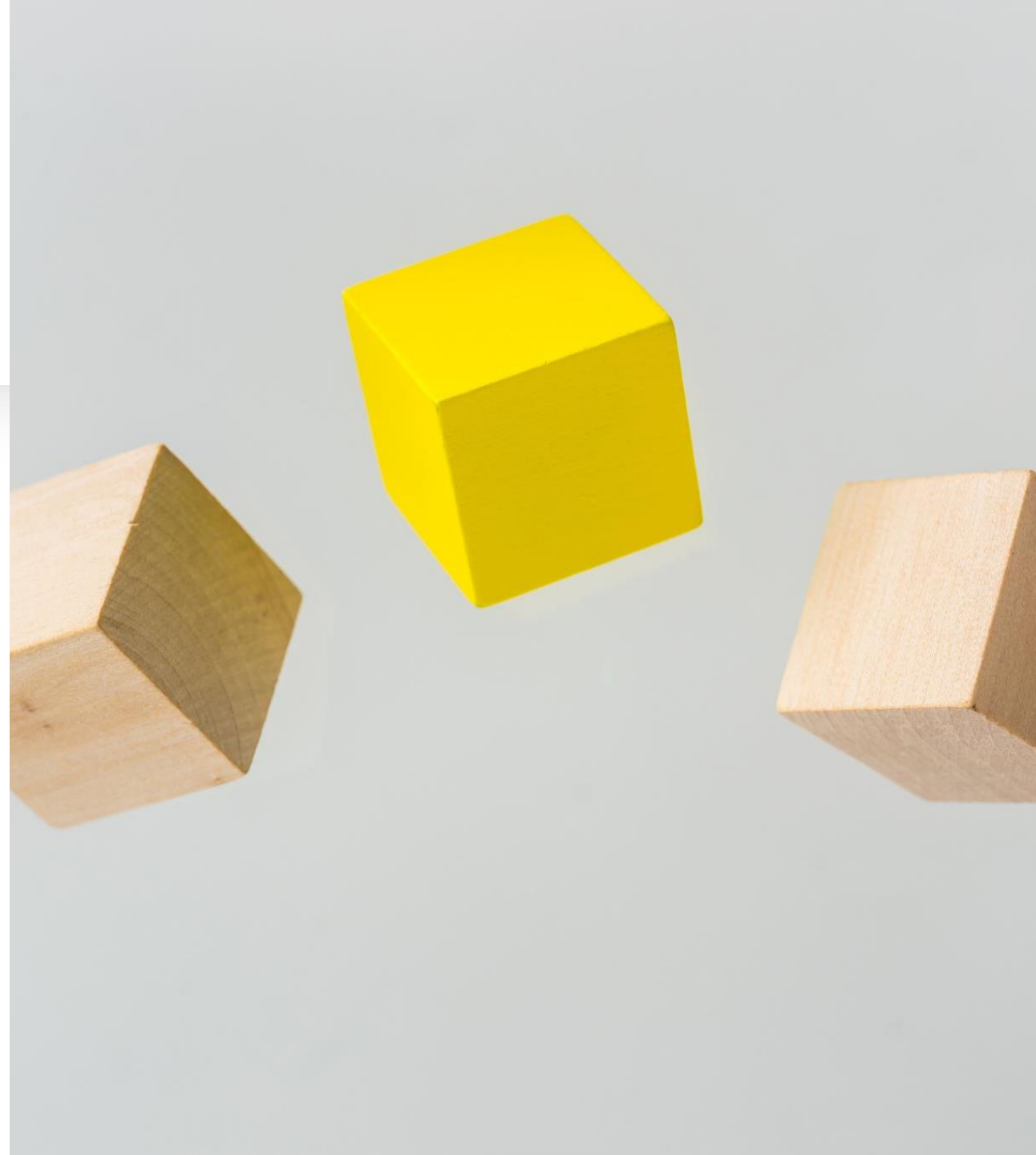
Domain Constraint

- Domain constraints ensure that the values entered into a column are valid and follow specific rules. They maintain **data integrity** by restricting the type, uniqueness, and validity of data.



Types of Domain Constraints

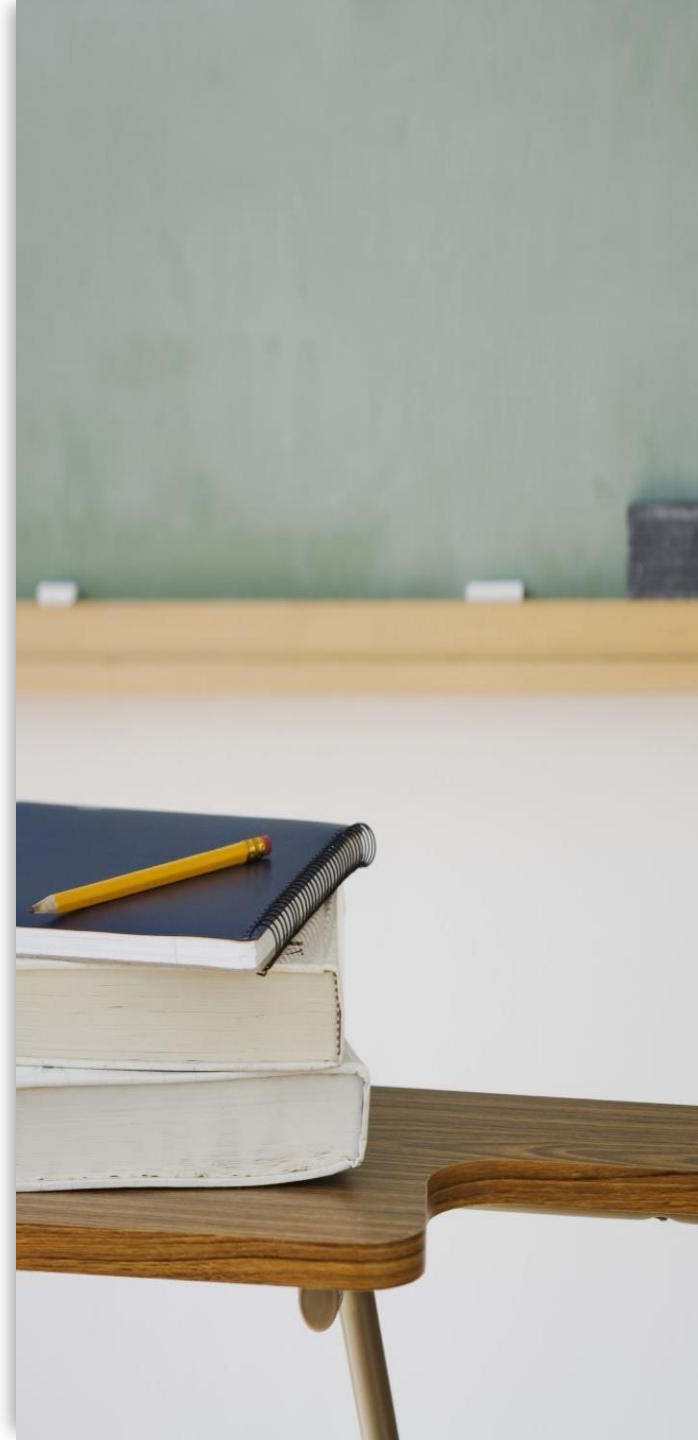
- NOT NULL Constraint
- UNIQUE Constraint
- PRIMARY KEY Constraint
- CHECK Constraint



NOT NULL Constraint

- Ensures that a column cannot have NULL values.
- Used when a field must always contain a value.

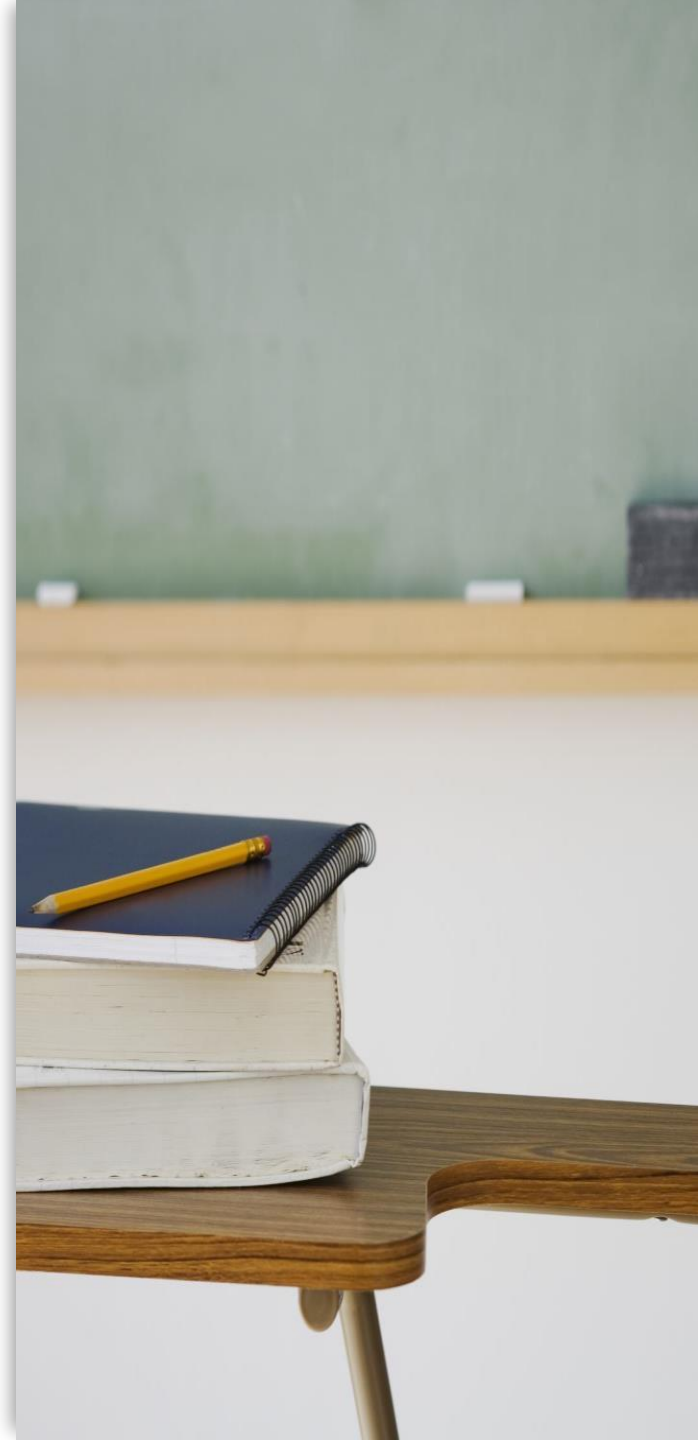
```
CREATE TABLE Student (  
    StudentID INT NOT NULL,  
    Name VARCHAR(50) NOT NULL  
);
```



UNIQUE Constraint

- Ensures that all values in a column are distinct.
- Allows NULL values (except when combined with NOT NULL).

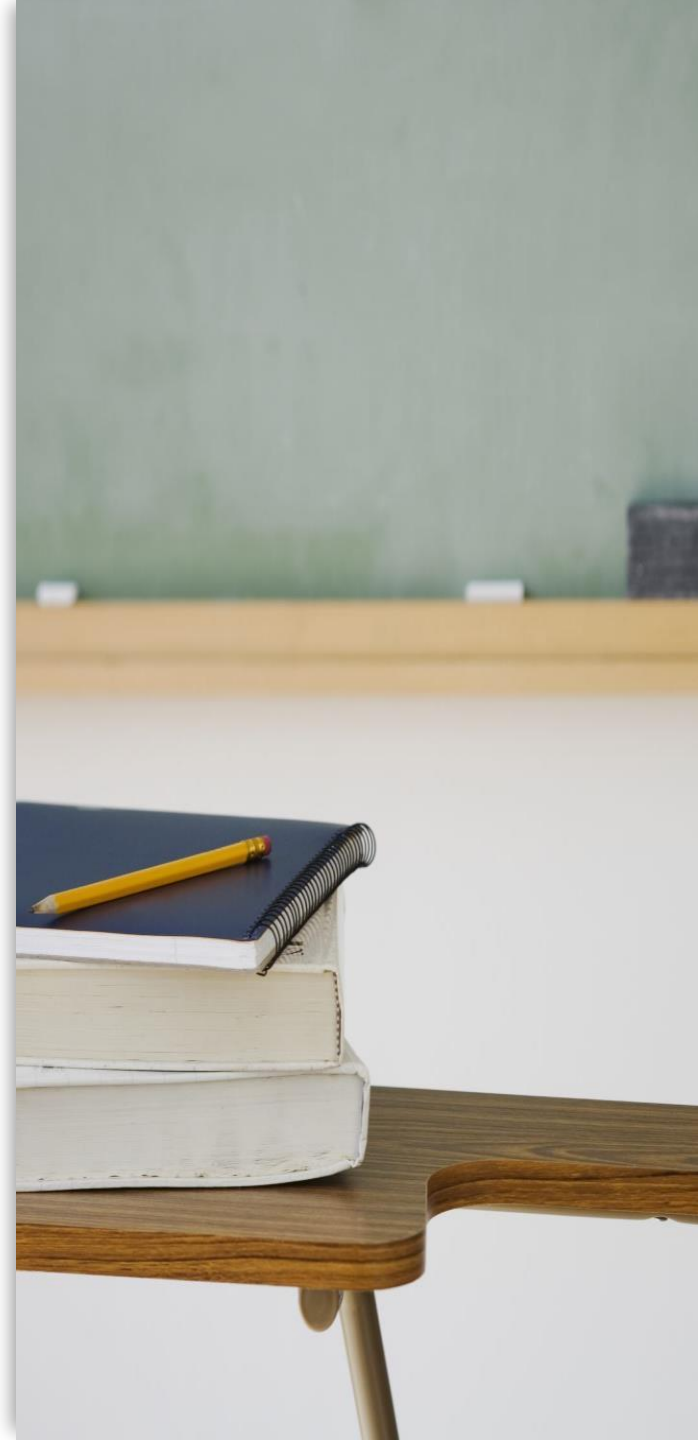
```
CREATE TABLE Employee (  
    Email VARCHAR(100) UNIQUE  
);
```



PRIMARY KEY CONSTRAINT

- Combines **NOT NULL** and **UNIQUE**.
- Identifies each row uniquely in a table.
- Only one primary key per table, but it can be composite (multiple columns).

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT NOT NULL  
);
```



A hand holding a black pen, writing on a document with a grid of numbers. The grid contains numbers 1, 2, 3, and 4 in a repeating pattern. The hand is positioned over the grid, and the pen is writing a number.

CHECK CONSTRAINT

- Ensures that values in a column satisfy a specific condition.

```
CREATE TABLE Product (  
    Price DECIMAL(10,2) CHECK (Price > 0),  
    Quantity INT CHECK (Quantity >= 1)  
);
```

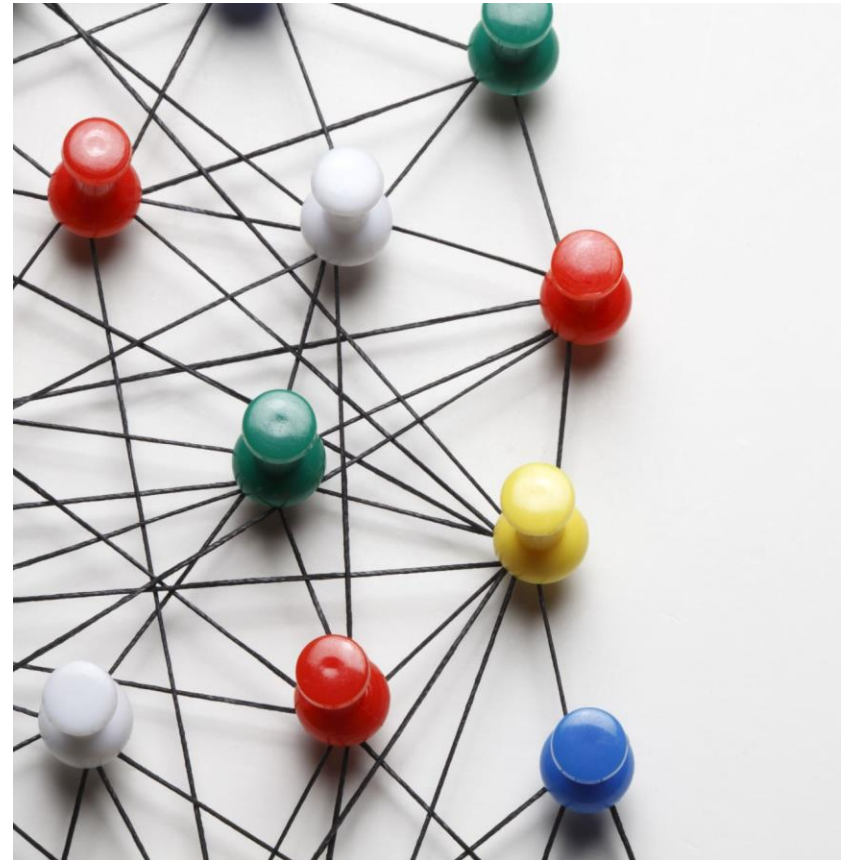
Domain Constraints

Constraint	Purpose	Allows NULL	Example Use Case
NOT NULL	Prevents missing values	 No	Student name must always be filled
UNIQUE	Prevents duplicate values	 Yes	Unique email addresses
PRIMARY KEY	Unique + Not Null identifier	 No	Order ID in Orders table
CHECK	Validates values with condition	 Yes	Price > 0, Age >= 18

These constraints are the **first line of defense** for ensuring clean, reliable, and consistent data in relational databases.

Referential integrity

- Ensures that relationships between tables remain consistent. It is enforced using **foreign keys**, which link a column in one table to the primary key of another table.
- **Definition:** A foreign key in one table must match a primary key in another table, or be NULL (if allowed).
- **Purpose:** Prevents invalid references and maintains logical consistency across tables.



Example

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50)  
);
```

```
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    DeptID INT,  
    FOREIGN KEY (DeptID) REFERENCES  
    Department(DeptID)  
);
```

Cascading actions

Cascading actions define what happens to dependent rows when a referenced row is updated or deleted.

1. **ON DELETE CASCADE**
2. **ON UPDATE CASCADE**
3. **ON DELETE SET NULL**
4. **ON DELETE SET DEFAULT**

NO ACTION / RESTRICT : Default behavior, block action if violation occurs .



ON DELETE CASCADE

- If a parent row is deleted, all child rows referencing it are also deleted.

FOREIGN KEY (DeptID) REFERENCES

Department(DeptID) **ON DELETE CASCADE**

Deleting a department will delete all employees in that department.





ON DELETE CASCADE

FOREIGN KEY (DeptID) REFERENCES
Department(DeptID) **ON DELETE CASCADE**

Deleting a department will delete all employees in that department.





ON UPDATE CASCADE

If the primary key in the parent table is updated, the foreign key in the child table is automatically updated.

```
FOREIGN KEY (DeptID) REFERENCES  
  Department(DeptID) ON UPDATE CASCADE
```

Changing a department's ID will update all employees linked to it.



ON DELETE SET NULL

If a parent row is deleted, the foreign key in the child table is set to NULL.

```
FOREIGN KEY (DeptID) REFERENCES Department(DeptID) ON DELETE SET  
NULL
```

Deleting a department will leave employees with DeptID = NULL.

ON DELETE SET DEFAULT

If a parent row is deleted, the foreign key in the child table is set to a default value.

```
FOREIGN KEY (DeptID) REFERENCES Department(DeptID) ON DELETE SET  
DEFAULT
```

Employees will be assigned to a default department if their department is deleted.

Cascades Behavior

Action	Behavior when Parent Deleted/Updated	Example Use Case
ON DELETE CASCADE	Deletes child rows	Remove employees when department is deleted
ON UPDATE CASCADE	Updates child foreign keys	Keep employee records consistent if DeptID changes
ON DELETE SET NULL	Sets child foreign key to NULL	Employee remains but without department
ON DELETE SET DEFAULT	Sets child foreign key to default	Assign employees to “General” department

Referential integrity + cascading actions ensure **data consistency** and prevent orphaned records in relational databases.

Assertions

- Assertions are **constraints at the database level** that enforce conditions across multiple tables, not just a single column or row.
- **Purpose:** Ensure complex business rules are always satisfied.
- **Scope:** Global (applies to the entire database, not just one table).



Example

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50),  
    Budget DECIMAL(10,2)  
);
```

```
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    DeptID INT,  
    Salary DECIMAL(10,2),  
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
);
```

```
CREATE ASSERTION dept_budget_check  
CHECK (NOT EXISTS  
    ( SELECT DeptID FROM Department d  
      WHERE d.Budget  
        < ( SELECT SUM(e.Salary) FROM Employee e  
          WHERE e.DeptID = d.DeptID)  
    )  
);
```

- This assertion checks **both tables** (Department and Employee).
- If any department's total salaries exceed its budget, the assertion fails.

Assertion

```
DROP ASSERTION dept_budget_check;
```

- **Assertions** are powerful because they can enforce rules involving **multiple tables**.
- **Constraints** (NOT NULL, UNIQUE, CHECK, etc.) are limited to a single table/column.

Triggers

Triggers are **procedures that automatically execute** in response to specific events on a table (INSERT, UPDATE, DELETE).

Purpose: Enforce rules, maintain logs, or perform automatic actions.

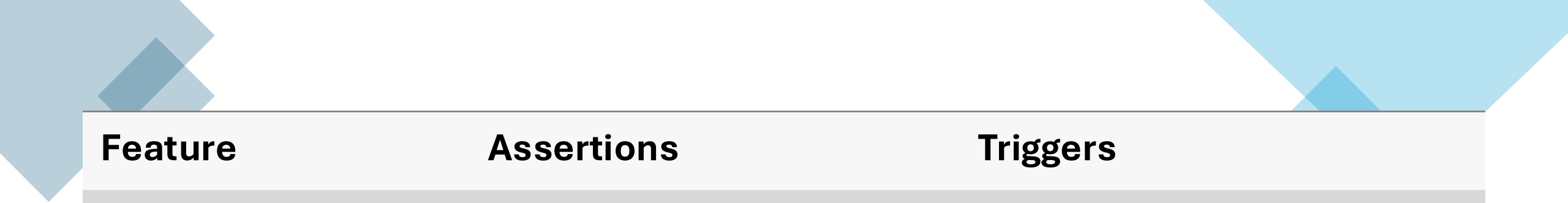
Scope: Table-level, event-driven.

Example

```
CREATE TRIGGER check_salary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF NEW.Salary < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salary cannot be negative');
    END IF;
END;
```

Example

```
DROP TRIGGER check_salary
```



Feature	Assertions	Triggers
Scope	Database-wide	Specific table & event
Purpose	Enforce global constraints	Automate actions on data changes
Execution	Checked when condition may be violated	Fires automatically on INSERT/UPDATE/DELETE
Example Use Case	“No employee salary < 0” across DB	“Log every deletion in Audit table”
Portability	Limited support in many DBMS	Widely supported in SQL systems

